

# FluxLink: A Unified Adaptive Platform for Secure Multi-Transport File Transfer

VE441 · App Development for Entrepreneurs · Project Proposal

Sun Qizhen   Zhang Jinbin   Li Yuhao   He Yumeng

UM-SJTU Joint Institute · Shanghai Jiao Tong University

2025

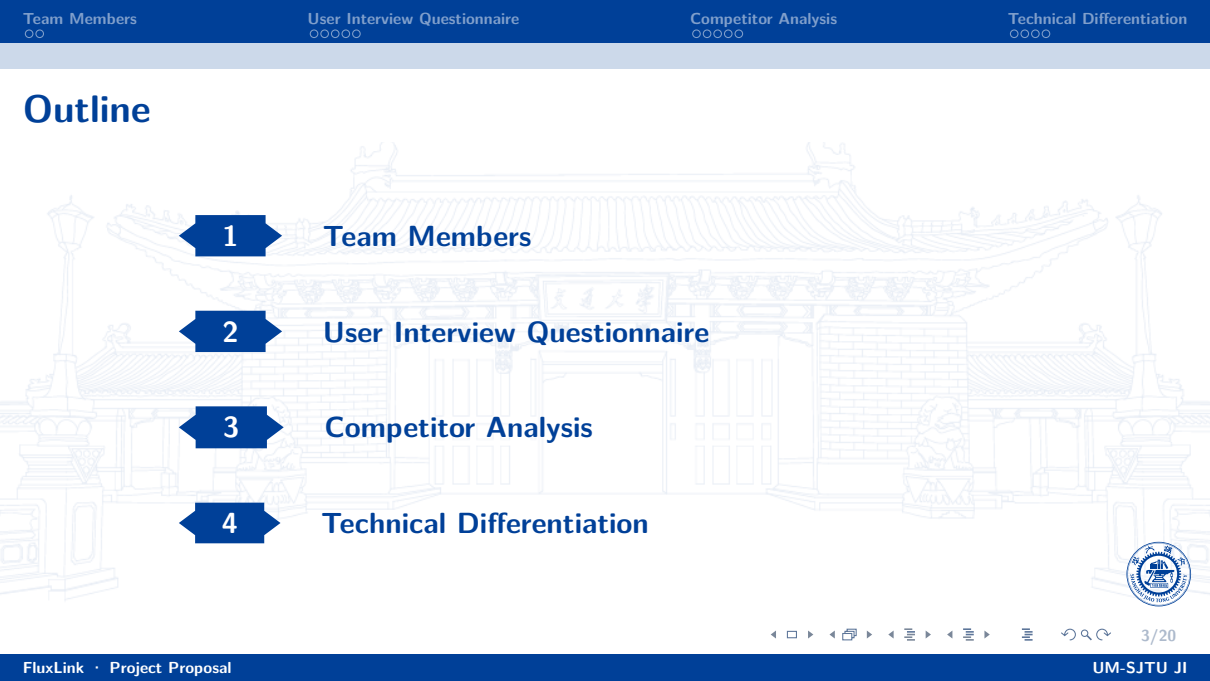
# FluxLink — Elevator Pitch

*Never waste time choosing how to send a file again —  
FluxLink automatically selects the fastest, most secure transfer path  
across any platform and any network.*

- **Zero decision cost** — routing is automatic
- **Any network** — LAN, P2P, relay fallback
- **Any platform** — Android, iOS, Desktop
- **Uniform E2EE** — all modes, all files
- **Encrypted chat** — context stays private
- **No account required** — pair via QR/token



# Outline

- 
- 1 Team Members
  - 2 User Interview Questionnaire
  - 3 Competitor Analysis
  - 4 Technical Differentiation



1

Team Members

2

User Interview Questionnaire

3

Competitor Analysis

4

Technical Differentiation



# Team Members

**Team Name:** NUUB

**PM:** Zhang Jinbin

| Full Name    | jAccount    | Task Assignment                                                       | Technical Strengths                                |
|--------------|-------------|-----------------------------------------------------------------------|----------------------------------------------------|
| Sun Qizhen   | qz_sun      | Product architecture · Protocol routing engine · Network path scoring | Networking · Distributed systems · System design   |
| Zhang Jinbin | moranxuege  | PM · Mobile client (Android/iOS) · Cross-platform UI/UX               | Android (Kotlin) · iOS (Swift) · UX implementation |
| Li Yuhao     | y4954he     | End-to-end encryption · QR/token handshake · Encrypted chat           | Cryptography · Backend security · Protocol design  |
| He Yumeng    | 17521218310 | Relay server · Deployment · Testing · Cache policy                    | Backend · DevOps · QA · Django REST                |

## Product & Research

Sun Qizhen  
routing logic &  
protocol priority

## Client & UX (PM)

Zhang Jinbin  
mobile UI &  
cross-platform

## Networking & Security

Li Yuhao  
E2EE model &  
handshake

## Backend & QA

He Yumeng  
relay infra & test  
coverage





# Interview Design — Participants & Venue

## How We Find Interviewees

- SJTU engineering/CS communities
- GitHub discussions, V2EX, WeChat tech groups
- Screening: regularly transfer files across multiple devices/OS, and have encountered at least one of: size limits, incompatibility, network failure, privacy concerns
- **Avoid friends/family** — reduce social desirability bias

## Venue & Session Structure

- Remote via Tencent Meeting / Zoom
- 10–20 min per session
- One interviewer + one note-taker
- Recorded with consent

|           |                                 |
|-----------|---------------------------------|
| 0–5 min   | Warm-up, device context         |
| 5–15 min  | Pain stories, failure scenarios |
| 15–20 min | Ideal workflow, retention       |



# Theme 1: Device Context & Current Behavior

*Goal: Establish devices/platforms used and current tools, without leading toward any product concept.*

- Q1** Walk me through the devices you use daily and how often they need to share files with each other.
- Q2** Think back to the last few weeks — what triggered your need to send a file, and to whom?
- Q3** What apps or methods do you normally reach for? How did you settle on those tools?
- Q4** When you're about to send a file, what goes through your head before deciding which tool to use?





## Theme 2: Pain Points & Failure Stories

*Goal: Surface real breakdown moments, workarounds, and emotional friction.*

- Q5** Walk me through the last time you sent a large file — from the moment you decided to send it to delivery.
- Q6** Has your usual method ever just stopped working? What happened, and what did you do instead?
- Q7** When transferring between iPhone and Android, or phone and a different-OS computer, how do you handle that?
- Q8** Describe a time when security/privacy crossed your mind during a file transfer. Did it change your behavior?
- Q9** What does it feel like when a transfer fails or arrives corrupted? Tell me about a specific moment.



## Theme 3: Motivations, Ideals & Accepted Friction

*Goal: Understand retention drivers, unarticulated needs, and normalized workarounds.*

- Q10** When a file transfer tool works really well, what makes you keep coming back to it?
- Q11** Describe your ideal file-sending experience for a scenario that comes up regularly for you.
- Q12** What frustrations have you accepted as “the way file transfer works” — things you’ve stopped trying to fix?



1

Team Members

2

User Interview Questionnaire

3

Competitor Analysis

4

Technical Differentiation



# Market Overview

## The Fundamental Gap

Existing systems typically optimize for **exactly one transport pathway**. No single consumer app combines all approaches into a unified, adaptive experience.

### Cloud Storage

Google Drive  
iCloud · Dropbox

### Messaging Apps

WeChat  
WhatsApp ·  
Telegram

### Ecosystem / LAN

AirDrop  
Nearby Share

### Internet P2P

Send Anywhere  
Snapdrop

### PAKE Transfer

Magic Wormhole  
croc

**Competitors analyzed:** Google Drive · WeChat · AirDrop · Send Anywhere ·  
Magic Wormhole



# Competitor Profiles (1/2)

## Google Drive — Cloud Storage

- + Persistent storage; cross-device sync; 15 GB free; no recipient install
- Double bandwidth (sender → server → receiver)
- Metadata leakage; subscription pressure; unsuited for transient transfer

## WeChat — Messaging App

- + 1.3B+ users; already in daily use; integrated cloud storage
- 100 MB limit; aggressive compression; server upload bottleneck
- Ecosystem lock-in; super-app surveillance risks

## AirDrop — Ecosystem LAN Transfer

- + Fast (Bluetooth + Wi-Fi Direct); no account; no internet required
- + Seamless Apple-to-Apple UX
- Apple-only; proximity-only; no fallback for remote scenarios
- Contact-discovery leaks phone numbers and email addresses to passive attackers (*Heinrich et al., 2021*)



## Competitor Profiles (2/2)

### Send Anywhere — Internet P2P

- + Cross-platform (Android, iOS, Web); 6-digit key; no account; 4.3🐼
- No intelligent routing between LAN/P2P/relay; relay-dependent in most real-world conditions; no resume on failure; no encrypted chat alongside transfer

### Magic Wormhole — PAKE Transfer

- + Strongest cryptographic model (SPAKE2); relay holds no plaintext; resists offline attack; open-source — *architecturally closest to FluxLink*
- CLI-only; no mobile app; no LAN fast path (all traffic routed through relay even on same Wi-Fi); no resume; single centralized relay



# Comparison Table

| Feature              | Google Drive | WeChat  | AirDrop | Send Anywhere | Magic Wormhole | FluxLink |
|----------------------|--------------|---------|---------|---------------|----------------|----------|
| Cross-platform       | ✓            | Partial | ×       | ✓             | ✓              | ✓        |
| LAN / local transfer | ×            | ×       | ✓       | ×             | ×              | ✓        |
| Internet / relay     | ✓            | ✓       | ×       | ✓             | ✓              | ✓        |
| Auto-routing         | ×            | ×       | ×       | ×             | ×              | ✓        |
| E2EE (all modes)     | ×            | ×       | Partial | Partial       | ✓              | ✓        |
| Encrypted chat       | ×            | ×       | ×       | ×             | ×              | ✓        |
| No account required  | ✓            | ×       | ✓       | ✓             | ✓              | ✓        |
| No ecosystem lock-in | ✓            | ×       | ×       | ✓             | ✓              | ✓        |
| Mobile app           | ✓            | ✓       | ✓       | ✓             | ×              | ✓        |
| Resume on failure    | ×            | ×       | ×       | ×             | ×              | ✓        |



1

Team Members

2

User Interview Questionnaire

3

Competitor Analysis

4

Technical Differentiation





# Intelligent Transfer Routing Engine

## Core Insight

Every competitor optimizes a **single transfer modality** and asks users to manage the rest manually. FluxLink treats the protocol stack as a **scored decision**, not a user configuration.

### Priority chain (probed automatically after pairing):

- ① **mDNS-based LAN** — lowest latency, highest throughput, no internet dependency
- ② **IPv6 direct P2P** — increasingly viable on campus and carrier networks
- ③ **QUIC/UDP hole-punching** — rendezvous-assisted NAT traversal; no file content on relay
- ④ **Centralized relay fallback** — guarantees delivery under any condition; relay handles ciphertext only

*The user sees one consistent interaction: **pair, send, done**. Path degradation triggers silent fallback.*



# Security Model & Encrypted Chat

## Uniform Security Across All Modes

- QR / short-code handshake establishes a session key (SPAKE2)
- All file chunks and chat messages encrypted **before leaving the device**
- Relay server never sees plaintext
- Relay-cached messages cleared weekly

## Encrypted Chat Layer

- **No competitor** integrates encrypted chat alongside file transfer
- The conversation about a file is as private as the file itself
- Same E2EE session — no context leakage

## Why This is Hard to Copy

The routing engine cannot be added incrementally to a single-modality app. Competitors would need to simultaneously rebuild their network layer, session model, and security architecture — at odds with their current business models and architectural philosophy.



# Summary

## What We Bring

- Intelligent multi-protocol routing engine
- Uniform E2EE across LAN, P2P, and relay
- Integrated encrypted chat
- No account, no ads, no decision cost

## Next Steps

- Conduct user interviews (10–15 participants)
- Synthesize pain points & refine problem framing
- Prototype routing engine core
- Validate pairing UX with paper prototypes

Thank you — Questions?





**Thank You**